

임베디드 SW 경진대회 · 최종 기술문서 (통합판)

불침번 (BulChimBeon)

상시 소방설비 점검 시스템 — 하드웨어 · 소프트웨어 · 앱 통합 기술문서

예선 제출 기준 · 하드웨어/소프트웨어 최종 기술문서 + 앱 개발 총정리 + 사업 확장성 자료 종합

작성일: 2026-08-27

기준 저장소: github.com/Billiardchalktheif/2026ESWContest_free_bulchimbeon (main)

참고 원본: 불침번_하드웨어_최종기술문서, 불침번_소프트웨어_기술문서(v7), 앱개발_총정리, 프로젝트설명_사업확장성_개선방향

본 문서는 팀 내 4개 자료(하드웨어 최종 기술문서, 소프트웨어 기술문서 v7, 앱 개발 총정리, 사업 확장성·개선방향 자료)를 하나의 최종 기술문서 형식으로 통합·재정리하고, 프로젝트 최초 구상안(착수 설계 v2) 대비 최종 결과물이 어떻게 달라지고 개선되었는지를 별도 장(10장)으로 정리한 것이다.

1. 프로젝트 개요

1-1. 문제의식과 설계 철학

법정 소방시설 점검은 연 1~2회 이루어지는 "특정 시점의 스냅샷(점)"인 반면, 실제 설비 열화는 점검과 점검 사이에도 연속적으로 진행되는 "선"의 과정이다. 불침번은 이 점과 점 사이의 공백을 상시 센서로 메우는 예측적 진단 시스템으로, 소방시설관리사의 업무를 보조하는 것이 목적이며 관리사를 대체하지 않는다 — 시스템은 이상 징후를 조기에 포착해 알리는 역할까지만 하고, 최종 판단과 조치는 사람이 담당한다.

설계 전반에는 소방공학의 Fail-Safe 원칙(Fail-Passive/Fail-Active/Fail-Soft)이 대입되어 있으며, 모든 센서 모듈은 인증 소방설비의 내부 회로·기능·구조를 변경하지 않는 비침습(non-invasive) 방식으로 부착된다. 이는 "인증설비를 임의로 개조"하는 데서 발생할 수 있는 소방시설법상 리스크를 원천적으로 차단하는 설계 방향이다.

1-2. 6 엣지 요약

엣지	대상 설비	핵심 기능	AI 적용
자탐1	차동식 열감지기(2계통 중 1)	루프저항 추세 + 온도상승률 기반 조기경보	—
자탐2	광전식 연기감지기	MQ-2·DHT22 기반 비화재보(오작동) 판별	O (RandomForest 5 클래스)
가스계	CO2 소화설비	로드셀 기반 용기 중량 열화 감시	—
수계	소화펌프(충압+주펌프)	H-Q 곡선 성능시험 + 전류파형 고장 분류	O (RandomForest, 충압펌프)
유도등	비상유도등	배터리 방전전압 감시	—
소화기	휴대용 소화기	가속도 기반 이동(이탈·도난) 감지	—

AI(머신러닝)는 프로젝트 전체에서 정확히 2곳에만 의도적으로 한정 적용되어 있다: ① 수계 충압펌프 전류 파형 분류, ② 자탐2(광전식) 비화재보 판별. 이 두 곳을 제외한 나머지 설비의 판정은 전부 통계 기반(선형회귀) 또는 규칙 기반(임계값)으로 구현된다. 두 AI 적용 지점의 공통점은 "여러 변수의 패턴을 동시에 봐야 구분되는 문제"라는 것으로, 이는 어디에 AI가 정말 필요한지를 설계 단계에서 선별한 결과다.

1-3. 시스템 구성도

ESP32 기반 센서 노드 7종(자탐1, 자탐2, 가스계, 수계, 유도등, 소화기 게이트웨이, 소화기 리프노드×4)이 라즈베리파이 4 중앙제어로 데이터를 전송하고, 서버가 규칙기반·회귀분석·AI 판정을 거쳐 정상/주의/경보 상태를 대시보드·LCD·부저로 출력한다. 예선(9/3 마감) 시점 구조는 자탐1·자탐2·수계·가스계·유도등 5개 스코프는 WiFi/UDP로, 소화기는 배터리 절약을 위해 ESP-NOW(무선) 리프노드 4개 + 게이트웨이 1개 구조로 각각 중앙제어에 데이터를 보낸다.

- 센서 노드(ESP32 7 종) → WiFi/UDP → 라즈베리파이(receiver: udp_listener.py → packet_parser.py)
- 패킷 해석 직후 곧바로 judge/*.py 판정 함수 호출 — 판정은 항상 실시간, 별도 배치 작업 없음
- 판정 결과 → storage(SQLite) 저장 + dispatch/lcd_buzzer_output.py(부저·LCD 경보) 동시 출력
- 대시보드(Flask, 5 초 자동 새로고침)와 현장 점검 앱(Flutter)이 같은 서버를 바라보는 별개의 클라이언트로 동작

통신 계층(소켓 수신)과 판정 로직을 물리적으로 분리한 것이 핵심 설계 원칙이다. 전송 방식이 바뀌어도(예: UDP→RS-485) 판정 로직 파일은 건드릴 필요가 없다.

2. 공통 설계 원칙 및 인프라

2-1. Fail-Safe 설계

원칙	정의	적용 현황
Fail-Passive	실패해도 기존 설비 기능을 방해하지 않음	전 모듈 비침습(tap) 방식으로 구현 — 배선·배관·기구 개조 없음
Fail-Active	실패를 침묵시키지 않고 능동적으로 알림	센서값 물리적 타당성 검사 (check_sensor_fault)로 "센서 고장"과 "화재"를 구분해 별도 경고. Heartbeat 통신두절 감지는 결선 단계 예정
Fail-Soft	일부 기능이 죽어도 핵심 기능은 하되 유지	ESP32 워치독 타이머(esp_task_wdt) — 센서 노드 7종(자탐1/2, 가스계, 유도등, 소화기 게이트웨이·리프노드, 수계) 전체 반영 완료. 소화기 리프노드는 딥슬립 로직을 제거하고 상시 polling 방식으로 재설계해 나머지 노드와 동일 패턴 적용

2-2. 비침습(tap) 방식

모듈	비침습 방식
유도등	배터리 단자에 병렬로 전압만 측정, 원래 회로는 그대로
수계	T피팅으로 압력만 겹다리 측정, 배관 흐름 자체에 개입 안 함
가스계	저장용기를 저울 위에 얹기만 함, 용기·밸브 구조 변경 없음
소화기	가속도센서 부착만, 소화 매커니즘엔 손대지 않음
자탐1/2	루프에 병렬 tap, 기존 수신기·감지기 회로 그대로

2-3. 점검모드(Test Mode)

실제 소방수신기의 "시험모드" 개념을 차용한 기능이다. 자탐1의 정상 동작 검증(부작동시험/작동시험) 처럼 실제 열원을 갖다 대는 물리적 시험을 할 때마다 진짜 화재 경보가 나가면, 시스템이 "실제 비상상황"과 "지금 점검 중"을 구분하지 못하는 문제가 있다. 판정 로직(evaluate_*)은 점검모드 여부와 무관하게 항상 동일하게 계산되며, 바뀌는 것은 판정 결과의 처리 방식(진짜 경보 vs 시험 기록)뿐이다. 노드에 달린 물리 텍스트스위치로 구현되어 통신 왕복 없이 5초 주기 UDP 패킷에 test_mode 필드 하나만 넣는다.

엣지	적용 방식	상태
자탐1	GPIO27 텍트스위치	구현 완료, 코드리뷰 통과
자탐2	GPIO27 텍트스위치(자탐1과 동일 번호, 다른 보드라 충돌 없음)	자탐1 패턴 이식 완료
소화기	게이트웨이 GPIO19 텍트스위치 1개로 리프노드 4개 전체 일괄 커버	결정 완료·구현됨
유도등	방전시험(s)/데모(d) 버튼 구현	방전시험 트리거(GPIO25 's')/GPIO26 'd') 자체가 이미 점검모드 역할 겸함
가스계	미구현	손실률 추세가 며칠 단위로 느려 순간적 오탐 리스크가 낮다고 판단, 낮은 우선순위로 보류
수계	이미 있던 성능시험 버튼(GPIO33, TEST_PERFORMANCE)이 참고 모델	해당 없음 — 애초에 순간 오경보 구조가 아님

2-4. 위치독 타이머(Fail-Soft) 및 견고화 패턴

무인 상시 감시 시스템 특성상, 센서 통신 오류나 라이브러리 버그로 loop()가 멈춰버리면(hang) 화재가 나도 감지를 못 하는 채로 방치될 위험이 있다. ESP32 하드웨어 태스크 위치독(TWDT)을 자탐1/2, 가스계, 유도등, 소화기 게이트웨이, 수계 6개 노드에 우선 반영해, loop()가 30초 안에 한 바퀴를 못 돌면 칩이 스스로 재부팅하도록 했다. 재부팅은 서버 입장에서 전원 재투입과 동일하게 처리되며(boot_id 갱신, seq 리셋), 서버 판정 로직은 이를 "재부팅 감지" 정보 로그로만 처리하고 오탐으로 잡지 않도록 이미 설계되어 있어 추가 수정이 필요 없었다. 소화기 리프노드는 당초 답슬립 구조라 이 패턴을 그대로 쓸 수 없었으나, 아래 재설계를 거쳐 결국 7개 센서 노드 전체에 동일한 위치독 패턴이 적용된 상태다.

- millis() 기반 논블로킹 루프 — delay() 최소화, 전송/버튼폴링/WiFi 재연결 등 여러 주기 작업 동시 처리
- WiFi 자동 재연결 — ensureWiFiConnected()가 주기적으로 상태 확인 후 재시도
- ArduinoJson + boot_id(esp_random()) + seq — 재부팅 감지, 패킷 유실 감지용 시퀀스 번호
- NTP 시각 동기화(getEpochSeconds) — 미동기 시 millis() 기반 폴백, 서버도 비정상 타임스탬프(ts<2023 년 기준)를 서버 수신 시각으로 대체
- StaticJsonDocument 크기 초과 시 doc.overflowed()로 조용한 필드 누락을 사전 감지

- 물리 버튼(텍스트스위치) 폴링+디바운스(30~50ms) 공통 패턴 — attachInterrupt() 방식은 "계속 눌린 것으로 오인식"되는 신뢰성 문제로 폐기됨

소화기 리프노드(ESP32-C3)는 당초 딥슬립 구조라 나머지 6개 노드와 다른 위치독 패턴이 필요했다 — loop()를 쓰지 않고 setup() 1회 실행 후 바로 딥슬립에 들어가는 구조라 "매 반복 reset()" 패턴이 안 맞았기 때문이다. ESP32-C3가 ext0/ext1 딥슬립 웨이크업 자체를 지원하지 않는 칩 하드웨어 제약으로 반복적인 API 오류가 발생하자, 이를 우회하는 대신 딥슬립 로직 자체를 제거하고 나머지 6개 노드와 동일한 상시 loop() 폴링 + 표준 위치독(esp_task_wdt) 패턴으로 재설계했다. 이 변경으로 리프노드도 나머지 6개 노드와 동일한 위치독 코드 패턴을 그대로 재사용할 수 있게 되어 구조가 통일되었으며, 위치독 반영과 딥슬립 API 버그는 모두 해소되었다. 다만 상시 폴링 방식은 딥슬립 대비 소모 전류가 늘어나므로, 배터리 운용시간 재산정은 결선 전 확인 필요 항목으로 남는다(§10-2).

3. 통신 아키텍처 및 서버 구조

3-1. 네트워크 인프라 — WiFi 안정화 이력

라즈베리파이 온보드 WiFi 칩(brcmfmac)을 AP로 여러 ESP32 클라이언트가 붙는 구조에서 연결 거부, WPA 협상 모호성, PMF(Protected Management Frames) 불일치, stale station 미정리 등 반복 장애가 발생했다. 개별 버그라기보다 이 구조 자체의 약점에서 파생되는 문제로 판단되어, 근본 대안을 다음 순서로 검토·적용한다.

우선순위	대안	리스크/신뢰도	비고
1 (즉시 권장)	외장 공유기 + 이더넷 유선 연결	낮음 / 높음	SSID-PASS-SERVER_IP만 변경, 코드 변경 거의 없음
2	USB WiFi Dongle로 온보드 칩 대체	중간 / 중간	칩셋: MT7612U·RT5370 권장, RTL8812AU/BU 비추천(ESP32 ASSOC_FAIL 확인)
3 (결선 이후)	ESP-NOW 중앙집중 리팩터링	큼(펌웨어 4종 재작성) / 높음	라즈베리파이가 상대하는 WiFi 클라이언트를 6→1개로 축소
4 (비추천)	BLE 전면 전환	매우 큼 / 불확실	동일 콤보칩(BCM43455)이라 다른 종류의 불안정 재발 우려

소화기 구간은 별도 문제였다. 게이트웨이가 라즈베리파이 핫스팟(6번 채널 고정)에 연결되는 동안, ESP-NOW 통신은 채널이 다르면 실패하는데 리프노드는 실제 AP에 연결하지 않고 ESP32 기본값인 1번 채널에 머문다. 여기에 WiFi STA 연결 중 기본 활성화되는 모뎀 슬립이 ESP-NOW 수신 순간과 겹치면 패킷이 유실된다. 해결책은 게이트웨이 쪽 채널 명시 지정(WiFi.begin(..., WIFI_CHANNEL)) + WiFi.setSleep(false), 리프노드 쪽 esp_wifi_set_channel() 강제 지정이나, 진단은 끝났고 해결 방안도 문서화되었지만 2026-08-24 기준 코드 반영은 여전히 미완료 상태로 통합테스트 전 최우선 확인 항목이다.

3-2. 서버 폴더 구조 (v7 재편)

v7에서 server/를 4개 서브패키지(receiver-storage-judge-dispatch)로 재편했다. 파일이 커지면서 "통신 계층인지, 판정 로직인지, 출력 채널인지"가 이름만으로 구분되지 않는 문제를 해결하기 위함이며, 재편 과정에서 로직 자체는 바꾸지 않았다(버그 3건 수정 제외).

- receiver/ — udp_listener.py(소켓 수신), packet_parser.py(패킷 해석·판정 호출)
- storage/ — db.py, schema.sql(5 개 설비 테이블 + heartbeat + 점검로그)
- judge/ — rules.py(소화기), regression.py(루프저항+가스계/유도등 회귀), differential.py(자탐 1 온도상승률), classify.py(AI 실시간 추론), nuisance_baseline.py
- dispatch/ — lcd_buzzer_output.py(부저+LCD 공통 출구)
- ml/ — 학습 스크립트 2 종 + 회귀 예측 정확도(MAE) 검증
- dashboard/, app_api/ — Flask 대시보드, Flutter 앱 연동용 FastAPI

3-3. 온라인/오프라인(Heartbeat) 판정 및 패킷 처리

설비마다 정상 전송 주기가 크게 달라(자탐/수계/소화기/유도등은 초 단위, 가스계는 실배포 기준 1일 간격) config.py에 device_type별 타임아웃을 별도 정의한다. is_node_online() 함수 하나를 packet_parser.py(서버 판정)와 dashboard/app.py(화면 표시)가 공유해 "서버는 온라인, 화면은 오프라인" 같은 불일치를 방지한다. 패킷 수신 시 upsert_heartbeat()로 boot_id·seq 기반 재부팅/유실 감지 후 device_type 분기 → 해당 테이블 INSERT → 즉시 evaluate_*() 판정 함수 호출 순으로 처리된다.

3-4. 경보 출력

모든 설비의 경보는 dispatch/lcd_buzzer_output.py의 trigger_alert() 하나를 거친다. 같은 (device, zone) 조합은 5분간 재알림하지 않으며, severity가 "caution"이면 LCD 표시만 하고 "alarm"일 때만 부저가 울려 주의 단계와 경보 단계를 소리로 명확히 구분한다. GPIO/LCD 하드웨어가 없는 개발 환경에서는 콘솔 로그로 자동 대체되어 개발 중에도 서버가 죽지 않는다.

4. 엣지별 상세 기술

4-1. 자탐 1 — 차동식구역

감지기의 형식승인 및 검정기술기준 제14조(차동식스포트형감지기 감도시험)에 따라, 1종 기준 10°C/min 이상 상승 시 작동, 2°C/min 이하 상승은 15분 지속해도 오작동 금지가 법적 근거다.

부품	인터페이스	핀/연결	비고
ADS1115(외장 ADC)	I2C	SDA→GPIO21, SCL→GPIO22	온도·루프저항 정밀 측정
TS0202 온도센서	Analog	ADS1115 채널 A0	10회 측정 후 중앙값 (median) 사용
루프저항(220Ω+가변저항 +100Ω 선트)	Analog	ADS1115 채널 A1	220Ω은 설계상 200Ω 대체(영향 미미)
점검모드 텍스트スイッチ	Digital In	GPIO27	디바운스 30ms
열풍기(화재 재현용 열원)	수동 조작	—	ESP32 미연결, 완전 수동

TS0202 raw ADC 값을 직접 판정에 사용하며(온도가 오르면 raw는 감소하는 NTC 음의 상관관계), 25초 이동창(5초×5샘플) 동안 raw가 500 이상 떨어지면 alarm이다. temp_c(섭씨 환산값)와 temp_rise_rate는 대시보드 표시용일 뿐 경보 판정에는 절대 사용하지 않는다 — 캘리브레이션 계수가 검증 전 placeholder일 수 있고 raw 단위 판정이 이미 충분한 마진을 갖도록 검증되었기 때문이다.

캘리브레이션 — 히스테리시스(열적 이력현상) 발견

작동시험 중, 같은 실측온도(23.5~23.6°C)인데도 가열 초반 raw≈9301, 냉각 후반 raw≈8111~8212로 1000 이상 차이가 확인되었다. 노이즈가 아니라 손 온도계와 임베디드 NTC 센서의 열용량·반응속도 차이로 인한 열적 이력현상으로 판단되어, 냉각 구간 데이터는 캘리브레이션에서 제외하고 가열(상승) 구간만 사용했다(TEMP_RAW_PER_C=-139.0, $R^2=0.932$).

마진 검증: 10°C/min(작동시험 기준)을 raw/20초창으로 환산하면 슬로프에 따라 약 326~464 raw/20초인데, 실제 열풍기 조사 시 관측된 raw 하락폭(5초 최대 4681, 20초 최대 7708)은 계산 임계값의 10~15배에 달해, 슬로프 추정 오차가 있어도 실화재를 놓칠 위험은 사실상 없는 것으로 확인되었다.

4-2. 자탐 1/자탐 2 공통 — 루프저항 2 층 판정

원래는 이동평균 기반 z-score 판정을 썼으나, baseline 자체가 서서히 상승하는 저항값을 따라가버려 진짜 "가속 열화"를 정상 범위로 오판하는 문제가 실측 검증 중 발견되어 폐기되었다. 대신 아래 2층 구

조로 재설계되었다.

- 1 층(절대 임계값) — 보정저항(raw-230)이 50Ω 을 초과하면 즉시 alarm(법정기준 그대로)
- 2 층(선형회귀 추세) — 최근 이력을 다운샘플링(100 개마다 1 개, 최대 50 개 표본)해 시간 대비 저항값을 선형회귀. 기울기가 8Ω/day 이상이면 alarm, 3Ω/day 이상이면 caution. 잔여 고장시간(RTTF)도 함께 계산
- 두 층 중 더 심각한 쪽(normal<caution<alarm)을 최종 상태로 채택

노이즈 시뮬레이션(5초 간격, 표준편차 0.3Ω, 10회×약 20.8시간)으로 steady-state 기울기 표준편차 약 0.45Ω/day, 99th percentile 약 0.92Ω/day를 실측하고, 이를 충분히 벗어나는 현재 임계값(alarm=8, caution=3)으로 확정해 오탐률 0%를 달성했다. "며칠~2주" 단위로 진행되는 실제 열화는 caution이 약 2.3일째, alarm이 약 6.4일째에 순서대로 안정적으로 잡히는 것도 시뮬레이션으로 확인되었다.

4-3. 자탐 2 — 광전식구역 (AI 적용 지점 ②)

MQ-2(가스/연기) + DHT22(온도·습도) 조합으로 화재/조리연기/평상시 세 가지를 동시에 구분해야 하는 문제라 RandomForest(5클래스: normal/smoking/incense/heat/fire, n_estimators=200, max_depth=6)를 적용한다. ML 모델 호출 전 "조리 연기" 시그니처(가스 상승 2% 이하 + 습도 상승 7%p 이상)를 규칙으로 먼저 걸러 모델을 부르지 않고 바로 cooking으로 확정한다 — 5클래스 모델이 학습한 적 없는 시그니처이기 때문이다.

원시 순간값이 아니라 기준점(baseline) 대비 변화량을 feature로 사용하며, 평상시엔 이동평균(rolling), 시연 중엔 대시보드 버튼으로 트리거한 고정 기준점을 쓴다. predicted_label=="fire"일 때만 실제 경보가 나가고 cooking/normal은 조용히 무시된다.

클래스	세션수	MQ2 상승률(평균)	습도 변화(평균)	온도 상승(평균)
normal	15	14.33%*	±0.24%p	+0.05°C
smoking	20	21.93%	+12.99%	+0.24°C
incense	10	37.61%	+3.07%	+0.41°C
heat	10	2.86%	-29.01%	+10.32°C
fire	9	23.85%	-35.94%	+13.06°C

1차 분리 기준은 습도 변화 방향(상승→normal/smoking, 하강→heat/fire)이며, 2차로 MQ2 반응 유무로 갈린다. incense는 습도 변화가 거의 없는 애매한 위치라 "MQ2만 단독으로 크게 뛰고 온도·습도는 안 움직인다"는 조합으로 별도 분리가 필요하다. 학습데이터 64세션 중 원인 미확인 이상치 2건(tobacco_09, incense_07)은 학습 데이터에서 제외하기로 결정하고 나머지 세션으로 모델을 재학습 완료했다.

4-4. 가스계 — CO2 소화설비

부품	인터페이스	핀/연결	비고
HX711(DOUT/SCK)	Digital	GPIO16 / GPIO17	로드셀 앰프 데이터/클럭
로드셀 A/B(10kg 바형×2, 병렬)	휘트스톤 브릿지	HX711 E+/E-/A+/A-	평판 양 끝 배치
TARE 텍트스위치	Digital In	GPIO18	영점조정 — 빈 상태에서 누름
SET_INITIAL 텍트스위치	Digital In	GPIO19	기준중량 저장 — 가스통 올린 직후

핵심 버그수정(v7): 손실률은 총중량이 아니라 "약제중량"(총중량-공병중량) 기준으로 계산해야 한다. 미니어처처럼 용기 자체 무게 비중이 크면 총중량 기준 계산은 실제 손실률을 과소평가해 임계치 판정이 늦어지는 문제가 있었다. 이 정정으로 10% 임계치 도달 시점이 "약 4.48일"에서 "약 2.1일"로 앞당겨졌다.

로드셀 신호선(흰/초록) 접촉불량으로 raw 값이 정상범위($\pm 650K$ 대)를 벗어나 -1.5M~-3.2M까지 튀는 문제가 발견되어, 신호선 납땜 조치 후 흔들림 테스트 편차가 $\pm 162g$ 수준에서 $\pm 16g$ 수준으로 개선되었다. 또한 에어컨 on/off에 따라 로드셀 판독값이 최대 $\pm 50g$ 요동치는 절대오차(영점 TARE 오차, 배율 오차 아님)가 발견되어 %손실 계산에서 상쇄되지 않는다는 점이 확인됐다 — 현재는 에어컨을 상시 가동해 온도를 안정시킨 구간만 분리 분석하는 방식으로 완화 중이며, 서미스터 기반 온도보상 로직은 미구현 상태다.

항목	값
회귀 적합도(R^2 , 학습구간)	0.951
기울기	-4.88 g/h (-117.0 g/day)
예측 도달 시점	청정구간 시작 +7.85h
실제 도달 시점	청정구간 시작 +8.67h
예측 오차	약 49분

최종 확정값: DEFAULT_CALIBRATION_FACTOR=87.9(납땜 후 501g 실측 기준), AVG_SAMPLES=30, 공병중량 260g(설치 현장 실측 필요), 손실률 임계값 CO2 10% / 할론·불활성가스 5%(팀 논의 끝 확정).

4-5. 수계 — 충압펌프 + 주펌프

부품	인터페이스	핀/연결	비고
INA219 #1(충압펌프, 0x40)	I2C+전류경로 직렬삽입	SDA→GPIO21, SCL→GPIO22	High-side, 전선 절단 후 삽입
릴레이 CH1/CH2(충압/주펌프)	Digital Out	GPIO25 / GPIO26	RELAY_ACTIVE_LOW=true
압력센서①(성능시험배관)	Analog(0.5~4.5V)	저항분압 → GPIO32	H-Q 곡선 핵심 계측
압력센서②(헤더)	Analog	GPIO34	충압펌프 자동제어 상시 감시용
유량센서(YF-S201)	Digital(홀센서 펄스)	GPIO27	성능시험배관에 직렬 삽입
성능시험 트리거	Digital In	GPIO33	니들밸브 잠근 직후 누름

가장 중요한 정정(2026-08-23): 원래 설계는 주펌프(main) 파형을 AI로 분류하는 것이었으나, 주펌프가 v13에서 INA219(전류센서)가 하드웨어 제약(A0/A1 패드 납땜 막힘)으로 제거되어 rms가 항상 NaN이 되면서 이 조건이 한 번도 실행되지 않는 죽은 코드였음을 뒤늦게 발견했다. 실제로 rms/peak/duty_cycle을 매번 보내는 쪽은 충압(jockey)펌프이므로, AI 추론 대상을 jockey로 정정했다. 같은 날 서버(receiver/packet_parser.py)의 AI 추론 트리거 조건이 `pkt.get("pump_type")=="main"`일 때만 `predict_pump_label()`을 호출하도록 되어 있던 버그도 함께 발견·수정했다 — "수계 파형 AI 학습 완료"는 맞는 말이었지만, 이 조건문 버그 때문에 대시보드에는 학습 완료 이후로도 AI 판정 결과가 표시되지 않고 있었다.

클래스	샘플 수	RMS 평균(mA)	RMS 범위	Peak/RMS 비율
dryrun	39	81.3	72.0~101.9	1.76
low_flow	867	182.3	113.8~245.0	1.52
normal	235	209.8	139.7~261.4	1.68
start_fail	156	7.3	0.9~68.7	1.97(변동 큼)

normal과 low_flow의 RMS 값은 83.7%가 서로 겹쳐(low_flow 867개 중 726개가 normal 범위 내) RMS 단일 지표만으로는 두 클래스 완전 분리가 어렵다는 점이 확인되었다 — Peak, Duty, Peak/RMS 비율(파형 형태)을 함께 쓰는 다변량 분류, 또는 유량·압력센서와의 결합이 보완책으로 제안된다.

주펌프 성능시험 및 H-Q 곡선

밸브는 니들밸브(수동 조작)로 ESP32가 직접 제어할 수 없어 압력값 패턴으로 서버가 밸브 상태를 추

정한다. 체절 상태에서 압력이 정격의 140% 초과, 또는 과부하 상태에서 65% 미만이면 법정기준 위반으로 alarm 처리된다. 성능시험용 축소 모델 배관은 실제 소방 배관 규격을 충족할 수 없다는 물리적인 한계가 있는데, 팀은 이 한계를 임계값을 낮춰 우회하는 대신

RATED_PRESSURE_KPA/RATED_FLOW_LPM을 펌프 스펙 기준 이론값(29.4kPa, 4.0L/min)으로 되돌려 축소 모델의 실측이 있는 그대로 불합격(체절압력 141.16% > 법정 140% 상한)으로 나오도록 확정했다 — "배관 규격 한계로 인한 불합격이며, 판정 로직 자체는 정상 작동"이라는 정직한 서사를 채택한 것이다.

헤더 압력을 300ms 주기로 상시 감시하며 히스테리시스 기준(START 10.0kPa, STOP 30.0kPa)으로 총 압펌프를 직접 기동/정지한다. 실측 검증 과정에서 v16(임계값 재조정)~v23(과다반복기동 판단 추가)까지 총 8차례에 걸친 버그 수정 이력이 있으며, 최종적으로 순간압력값 하나만이 아니라 지속시간 조건과 반복기동 횟수를 함께 보는 다층 판단으로 안정화되었다.

4-6. 소화기 — 이탈/도난 감지

설계 변경 이력: 원래는 압력게이지 방식이었으나 게이지 지름이 30mm로 물리적 센서 부착이 어렵고, 카메라 방식은 대량 배포 시 원가가 "저비용 상시점검" 취지에 맞지 않아, 가속도+게이트웨이 연결상태 기반 이탈/도난 감지로 전면 교체되었다.

부품	인터페이스	핀/연결	비고
MPU6500(SDA/SCL/INT)	I2C / 인터럽트	GPIO8 / GPIO9 / GPIO2	모션 인터럽트 — 상시 polling과 병행 감시
게이트웨이 ESP-NOW 수신	무선	—	리프노드 4개로부터 가속도 이벤트 수신
게이트웨이 WiFi/UDP 송신	무선	—	라즈베리파이로 중계 전송

리프노드(ESP32-C3, MPU6500 가속도)가 이동/충격을 감지해 ESP-NOW로 게이트웨이(ESP32 DevKitC)에 전송하고, 게이트웨이가 WiFi/UDP로 라즈베리파이에 중계한다. 전력 설계 변경: 당초에는 배터리 절약을 위해 딥슬립(1시간 주기 타이머 웨이크업 + MPU6500 모션 인터럽트 웨이크업) 구조를 채택했으나, ESP32-C3가 ext0/ext1 딥슬립 웨이크업을 애초에 지원하지 않는 칩 하드웨어 제약으로 반복적인 API 오류가 발생했다. 이를 우회 API(esp_deep_sleep_enable_gpio_wakeup())로 고쳐 쓰는 대신, 딥슬립 로직 자체를 제거하고 나머지 6개 노드와 동일한 상시 loop() 폴링 방식으로 재설계했다 — 위치독(Fail-Soft) 패턴도 이 변경으로 나머지 노드와 통일 적용이 가능해졌다. 이 결정의 트레이드오프는 상시 폴링에 따른 배터리 소모 증가이며, 결선 전 배터리 운용시간 재산정이 필요한 항목으로 관리된다.

2단계 판정: 1단계(evaluate_tamper)에서 가속도가 임계값(15.0m/s^2)을 넘으면 status="moved"로 표시만 하고 아직 alarm은 아니다. 2단계(resolve_stale_tamper_events)에서 10초(CONFIRM_WINDOW_SEC)가 지나도록 해당 노드의 후속 패킷이 없으면 진짜 이탈로 판단해 status="missing"+alarm 처리하고, 후속 패킷이 왔으면(단순 접촉·청소 등 오탐) normal로 되돌린다. v2→v3에서 "바로 다음 패킷 하나만 확인"에서 "순수 시간 기반 스윙" 방식으로 바뀌었는데, 리프노드가 딥슬립에서 200ms 폴링 방식으로 바뀌면서 예전 가정(다음 패킷이 금방 온다)이 깨졌기 때문이다. 이탈 확정 후 수동 복구 기능은 보안경보 성격이라 사람이 직접 확인 후 처리해야 한다는 설계 판단으로 의도적으로 미구현이다.

알려진 이슈(최우선): 딥슬립 웨이크업 API 오류와 리프노드 위치독 미반영은 딥슬립 로직 제거 + 상시 폴링·표준 위치독 적용으로 해소되었다. 다만 ESP-NOW/WiFi 채널 불일치 및 모뎀슬립 간섭 해결 코드(WIFI_CHANNEL 명시 지정, WiFi.setSleep(false), esp_wifi_set_channel() 강제 지정)는 게이트웨이·리프노드 양쪽 모두 아직 반영되지 않아, 통합테스트 전 최우선 확인 대상으로 남아 있다.

4-7. 유도등 — 배터리 전압 감시

필드	질문	계산 근거	단위	갱신 시점
predicted_days_to_replace	배터리가 언제 교체 필요하냐	평상시 전압 추세의 느린 선형회귀	일	상시
estimated_discharge_min	지금 정전나면 몇 분 버티는냐	데모모드(실제 방전시험) 구간 실측	분	방전시험 실행시에만

두 값은 절대 서로 비교되거나 합쳐지지 않는다 — 질문 자체가 다르기 때문이다. 착수 설계 문서(v2)의 CD74HC4067 16채널 멀티플렉서 구조는 최종적으로 채택되지 않았고, 4개 배터리를 ADC1의 개별 GPIO(32/33/34/35)에 직결하는 구조로 단순화되었다 — WiFi 사용 중에도 안정적으로 읽히는 ADC1 채널만 사용하는 제약과 맞물린 결과다.

충전시간	300mV대 진입	200mV대(기능저하) 진입	100mV대 진입	20분 시점 전압	법정기준(20분)
1h	145초(2.4분)	167초(2.8분)	199초(3.3분)	—	미충족
2h	425초(7.1분)	477초(8.0분)	557초(9.3분)	—	근소 미충족
6h	503초(8.4분)	1511초(25.2분)	2231초(37.2분)	264.9mV	300mV 기준 근소 미달
8h	1446초(24.1분)	2107초(35.1분)	3015초(50.2분)	312.1mV	충족

200mV대는 육안 관찰상 이미 "기능이 떨어지는 수준"으로 확인되어 사전예측 시스템의 정상/경보 경계로 쓰기엔 이미 늦은 지점이다. 300mV대를 "어둡지만 아직 인식 가능"한 마지막 정상 기능 구간으로 판단해 EVAC_DISCHARGE_THRESHOLD_V=0.30V로 채택했으며, 200mV는 폐기하지 않고 2차(이미 늦은) 경보 보조 임계값으로 유지한다. 서로 다른 개체 3대에서 모두 70~78mV 범위로 재현된 "깜빡임 시작" 지점은 충전량·개체차와 무관한 이 유도등 모델 승압회로의 물리적 최소 동작전압으로 판단되었으나, 이미 기능 상실에 근접한 시점이라 조기경보 기준으로는 채택하지 않았다.

5. AI 적용 지점 요약

#	위치	알고리즘	입력 Feature	출력 클래스	학습 스크립트
①	수계층압 펌프 파형 분류	RandomForest	rms, peak, duty_cycle	normal/dryrun/low_flow/start_fail	ml/train_pump_classifier.py
②	자탐 2비화재보판별	RandomForest (n_estimators=200, max_depth=6)	temp_rise_rate, mq2_raw, humidity_pct, mq2_to_humidity_ratio	normal/smoking/incense/heat/fire (5클래스)	ml/train_nuisance_classifier.py

두 지점 모두 "성능시험/규칙기반 필터를 먼저 거치고, 거기서 안 걸리지는 애매한 경우에만 AI가 최종 판단"하는 순차 구조가 아니라, AI는 항상 상시로 독립 실행되고 규칙기반 결과가 있을 때만 사후 대조하는 병행 구조다. ml/evaluate_predictions.py는 회귀 기반 예측(가스계/유도등의 predicted_days_*)에 대해서만 MAE 기반 정확도 검증(70:30 분할)을 수행하며, 방전시험 실측값(estimated_discharge_min)은 방법론이 달라 검증 대상에서 명시적으로 제외되어 있다.

6. 대시보드

Flask 서버사이드 렌더링 + 5초 간격 자동 새로고침(JS setInterval) 방식의 단일 페이지다. 입력 필드(달력 등)에 포커스가 있는 동안은 새로고침을 일시정지해 입력 도중 화면이 리셋되는 문제를 방지한다. 설비별로 최신 1건씩(GROUP BY node_id + MAX(id))을 카드로 표시한다.

- 자탐 카드 — raw 저항값이 아니라 보정값(raw-230)을 기본 표시. 실제로 법정기준(50Ω)과 비교되는 게 보정값이지 raw 가 아니므로, raw 를 그대로 보여주면 관리사가 착오를 일으킬 위험이 있기 때문. raw/오프셋/계산식은 <details> 토글로 숨김

- RTTF(잔여 고장시간) 표시 — 법정기준을 이미 초과한 상태에서 값이 음수로 계산되면 "마이너스 며칠"이 아니라 "이미 기준 초과" 문구로 명시적 대체
- 수계 H-Q 곡선 — 유량센서 실측값 도입(2026-08-23) 이후, 목표 3 점만 잇는 대신 밸브를 열어가는 전체 실측 궤적을 그리고 정격운전점은 "도달하지 못한 목표"로 별도 마커(삼각형) 표시
- 소화기 카드 — 내용연수 만료일 직접 입력 폼 내장, AJAX 저장 후 새로고침
- 자탐 2 카드 — "시연 시작" 버튼으로 그 시점을 고정 기준점(baseline)으로 등록해 짧은 시연 구간에서도 판별이 잘 동작하도록 함

7. 현장 점검 앱

7-1. 왜 앱이 필요했는가

방재실 대시보드는 "누군가 상시로 화면을 지켜보는 관제 거점"을 전제로 설계됐다. 하지만 실제 소방 시설 점검은 관리사가 건물을 돌아다니며 개별 설비를 눈으로 확인하고 점검표를 작성하는 방식으로 이뤄진다 — 방재실 인원은 한 화면에서 6개 설비 전체 상태를 동시에 파악해야 하는 반면(대시보드), 현장 점검인력은 지금 눈앞의 설비 하나에 대한 정보가 필요하고 점검했다는 사실을 기록·증빙해야 한다(앱). 앱은 대시보드의 축소판이 아니라, "현장에서 들고 다니며 개별 설비를 점검하고 그 행위를 기록·증빙하는 도구"로 역할이 다르다.

	방재실 대시보드	현장 앱
사용 주체	방재실 상주 관리자	순회 점검인력
화면 구성	6개 설비 전체 동시 표시	개별 설비 단위 진입(QR/리스트)
데이터 방향	읽기 전용	대부분 읽기 + 점검완료 시에만 쓰기
핵심 기능	AI 열화 추세, 실시간 판정	점검 수행 기록, 이력 PDF 출력
접근 경로	상시 접속된 모니터	QR 스캔 또는 앱 내 리스트
존재 목적	관제·조기경보	법정 점검 업무 보조·증빙

두 시스템은 같은 라즈베리파이 서버를 바라보는 별개의 클라이언트다. 앱과 대시보드가 서로 직접 통신하지 않고 파이가 유일한 중앙 서버 역할을 하며, 각 클라이언트는 파이의 API만 호출한다. 이 구조 덕분에 대시보드 쪽 기존 코드(센서 판정 로직, AI 모델)를 한 줄도 건드리지 않고 앱 기능을 독립적으로 엮을 수 있었다.

7-2. 완성 기능 (예선 제출 기준)

- QR 스캔 — 앱 내장 카메라로 설비별 QR 인식 → 해당 설비 상세화면 진입
- 설비 상세화면 — 대시보드 화면(WebView)을 그대로 로드해 실시간 센서값·AI 판정 확인
- 점검완료 기록 — 버튼 한 번으로 점검자·시각·상태를 서버에 저장
- PDF 리포트 — 그날 점검완료된 설비들을 모아 통합 리포트로 출력, 점검완료→PDF 반영까지 실제 데이터로 연동 확인 완료
- 홈 화면 체크리스트 — 6개 설비 상태와 점검 여부를 한 리스트에서 확인(현재 상태값은 더미 데이터)

개발 과정에서의 주요 판단

- QR 접근 제어 — URL 딥링크(커스텀 스킴) 대신 앱 내장 QR 스캐너(mobile_scanner)를 그대로 활용. 일반 카메라 앱으로 찍으면 텍스트만 인식될 뿐 앱과 연결되지 않아, 별도 스킴 등록 없이도 "앱을 통해서만 유효하게 동작한다"는 요구 충족
- 대시보드 통합 vs 분리 — 설비별 새 라우트 신설 대신, 기존 통짜 대시보드 화면을 재사용하고 앱이 그 위에 "현재 보고 있는 설비" 배너와 점검완료 버튼만 있는 방식으로 확정. 대시보드(Flask) 코드는 한 줄도 수정하지 않음
- 코드 격리 원칙 — 앱 연동용 API 는 기존 센서/판정 로직과 완전히 분리된 새 파일(inspection_api.py)로 작성

7-3. 결선 발전방향 (미완성 항목)

- 앱 아이콘 커스텀 이미지 — 현재 Flutter 기본 아이콘 사용 중
- PDF 한글 폰트 미적용 — 현재 PDF 출력 시 한글이 깨짐(데이터 자체는 정상 반영 확인)
- 점검이력 영구 저장 미비 — 현재 서버 메모리에만 저장되어 재시작 시 소실. 실서비스 전환 시 DB 저장 및 날짜범위 조회 제한 필요
- 홈화면 실시간 자동 갱신 부재 — 현재는 화면 진입/탭 전환 시점에만 데이터 재조회
- QR 의 역할 제한적 — 대시보드에 설비별 필터링 라우트가 없어 QR 과 홈화면 리스트 진입이 동일 화면으로 연결됨. 결선에서 /equipment/{id} 라우트 추가 시 QR 이 진짜 필터링 진입점으로 격상
- [결선 최우선] 홈화면 상태 표시가 실제 DB 와 불일치 — 체크리스트 API 가 하드코딩된 더미 상태값을 반환 중이라, 실제로는 경보 상태인 설비가 앱에서는 정상으로 표시되는 경우 확인됨. 대시보드와 동일한 DB 를 조회하도록 연동 필요

8. 사업 확장성 및 개선방향

8-1. 확장 우선순위 — 엿지 추가보다 6 엿지 심화가 먼저

소방시설은 설비마다 별개의 화재안전기준(NFSC/NFPC)이 적용되므로, 새 엿지(스프링클러·제연설비 등)를 추가하면 처음부터 법정 기준 조사와 캘리브레이션 실험을 다시 쌓아야 한다. 판매 논리의 핵심은 "커버리지"가 아니라 "설비당 예측 신뢰도"다. 6엿지는 일반 건물에 통상 함께 존재하는 소방설비 조합이라 "한 시스템으로 건물 소방설비 전체를 커버한다"는 완결된 패키지 서사가 성립하며, 확장은 임계값·정확도가 확정된 이후 구체적 수요에 따라 수요 주도로 진행한다.

8-2. 시장 세그먼트 전략

- 신축 건물(1 차 타겟) — 배관·배선 공사가 진행되는 시점에 RS-485 통신선을 함께 포설하면 한계비용이 낮음
- 기존 건물/레트로핏(2 차 타겟) — 전면 재배선 비용이 진입장벽이므로 배터리 기반 무선 서브노드+유선 전원 게이트웨이 조합의 무선 옵션 상품을 별도 제공. "유선이 기본 스펙, 무선은 레트로핏용 옵션"으로 이원화
- 비용 프레이밍 — 재배선 비용은 일회성 초기 투자(CAPEX), 법정 정기점검 인건비·화재사고·보험료는 반복 지출. 점검 인건비 절감분으로 회수되는 ROI 서사로 재구성

8-3. 결선(11 월) 확장 로드맵 — 통신·센서 인터페이스 이원화

축	예선(현재)	결선(사업화)
통신	무선(WiFi/ESP-NOW)	RS-485 유선(리프노드만 무선 예외 유지)
센서 계측	일부 침습(인라인 센트, DC 테스트베드 한정)	비접촉(CT클램프 — 실제 소방펌프는 AC 구동)
엿지 범위	6엿지 프로토타입 검증	6엿지 심화 → 수요 주도 확장

자탐1/2, 수계, 가스계, 유도등, 소화기 게이트웨이는 전부 RS-485(Modbus RTU) 유선 버스로 전환한다. Wi-Fi/ESP-NOW는 콘크리트 벽·철근 감쇠로 방재실까지 직접 도달을 보장할 수 없어 구역마다 중계기가 필요해지고 그 중계기 자체가 전원·설치·배터리 관리 대상이 되는 구조적 문제가 있는 반면, RS-485는 데이지체인 버스로 최대 1.2km까지 안정적으로 통신 가능해 중계기 문제 자체가 사라진다. 실제 R형 수신기 시스템도 수신기-중계기 간 유선 차동신호가 업계 표준이며, 소화기 리프노드만은 이동 감지가 핵심 기능이라 구조적으로 무선이 필수이므로 결선에서도 100% 무선(ESP-NOW)을 유지한다.

센서 인터페이스도 침습적 계측(테스트베드)에서 비접촉 CT클램프 계측(사업화)으로 전환한다. 예선 단

계의 인라인 센트(INA219) 채택은 테스트베드가 소형 DC 워터펌프를 쓰기 때문에 생긴 제약(트랜스포머형 CT클램프는 DC를 측정할 수 없음)일 뿐, 실제 소방펌프(3상 380V 또는 단상/3상 220V 유도전동기 AC 구동)의 전기적 특성과는 무관하다는 점을 명확히 구분해 서술한다. 사업화 버전에서는 저렴하고 검증된 비접촉 CT클램프로 충분하다.

8-4. 6 엣지 비접촉 전환 방안 (일관성 확보)

엣지	계측 원리	비침습 방식
유도등	배터리 단자 전압	고임피던스 전압분배기 병렬 탭 — 전류를 거의 끌지 않아 인 증설비 방전 동작에 영향 없음. 이미 적용 완료
수계	펌프 전류 파형	CT클램프 비접촉 — 실제 소방펌프는 AC 구동이라 적용 가능
자탐1(1단계)	루프저항	기존 단자대에 탈부착형 클립 프로브 상시 연결 — 법정 정기 점검 표준 관행과 동일한 방식
자탐1(2단계, 로드맵)	접촉저항 열화	완전 비접촉 IR 온도센서(MLX90614 등)로 단자대 국부 발열 감지. 실측 검증 필요한 차세대 방향
가스계	용기 중량	로드셀 외부 거치 — 이미 비접촉/비침습

8-5. Q&A 및 부가 사업 모델

- Q. 무선이면 사업화하기 쉽지 않나? → 벽 투과 실패로 중계기가 필수가 되고, 중계기 자체가 관리 부담이 되는 구조적 결함이 있음. RS-485 는 이 문제 자체를 해소
- Q. 재배선 공사비가 더 크지 않나? → 신축은 한계비용 낮음, 기존 건물은 무선 옵션 이원화, 비용은 CAPEX 대 반복 지출(점검 인건비) 관점의 ROI 로 재구성
- Q. 침습적 결선(전류 센트)은 마개조 아닌가? → 예선은 데이터 수집용 테스트베드임을 명시하고, 결선에서 비접촉 센서로 전환할 로드맵을 제시("문제 인지+해결 방향 설계")
- 참고 — 반복 수익 모델(SaaS/HaaS): 하드웨어를 설치비 수준으로 낮게 제공하고 대시보드·AI 알리를 구독료로 받는 구조
- 참고 — 데이터 네트워크 효과: 여러 건물의 열화 데이터가 누적될수록 AI 분류기 정확도가 개선되어 후발주자 대비 진입장벽으로 작용 가능

9. 최초 구상안 대비 변화 — 무엇이 얼마나 달라지고 개선되었나

아래 내용은 착수 단계 설계문서(v2, 조감도·핀배치 기준)와 초기 코드베이스를 기준으로 한 "최초 구상안"과, 2026-08-24~27 시점 실측·디버깅을 마친 "최종 구현"을 항목별로 대조한 것이다. 변화의 대부분은 아이디어가 틀려서가 아니라, 실물 하드웨어로 검증하는 과정에서 이론적 가정과 실측 결과 사이의 간극(hysteresis, DC/AC 전류 측정 원리, 온도 드리프트, 죽은 코드 등)을 발견하고 근본 원인까지 추적해 재설계한 결과라는 점이 공통적이다.

9-1. 핵심 변경 사항 종합표

항목	최초 구상(v2)	최종 구현	변경 사유 · 의의
수계 전류 센싱	CT클램프(SCT-013, 비접촉)	INA219(I2C, 션트저항 직렬 삽입)	CT클램프는 전자기 유도 방식이라 DC 전류(테스트베드 워터펌프 12V DC 구동)를 근본적으로 측정할 수 없음을 실측 전 이론 검토 단계에서 발견 — 설계 오류를 사전 차단
유도등 배터리 읽기	CD74HC4067 16채널 멀티플렉서	배터리 4개를 ADC1 개별 GPIO에 직결	개별 ESP32 원가 절감 목적의 멀티플렉서 구상을 실제로는 배선 단순화·안정성 우선으로 재설계. 분압 없이 1kΩ 보호저항만 직렬
소화기 감지 방식	압력게이지 광학 판독(카메라/센서)	가속도(MPU6500) + 게이트웨이 연결상태 2단계 판정	게이지 지름 30mm·공차 4mm로 물리적 센서 부착이 불가능함을 실측으로 확인. 목표 자체를 "압력 판독"에서 "이탈·도난 감지"로 재설정 — 저비용 대량배포 취지에도 더 부합
수계 AI 추론 대상	주펌프(main) 파형 분류	충압펌프(jockey) 파형 분류로 정정	주펌프 INA219가 v13에서 하드웨어 제약으로 제거되어 rms가 항상 NaN — 실제 파형을 보내는 쪽이 충압펌프임을 뒤늦게 발견. 서버측 트리거 조건 버그(main 전용)까지 함께 수정해 "학습은 됐는데 대시보드에 안 뜨던" 문제 해소

항목	최초 구상(v2)	최종 구현	변경 사유 · 의의
통신 아키텍처	5개 스코프 전부 WiFi/UDP 무선 구상	예선: 무선 유지 + 소화기 게이트웨이 USB 유선 선제 실증 / 결선: 전면 RS-485 유선 + 리프노드만 무선 예외	라즈베리파이 AP 반복 장애 (WPA 협상, PMF 불일치)를 실측 중 발견 → "이동이 필요한 노드와 필요 없는 노드를 기능 요구사항에 따라 재배포"하는 설계 판단으로 발전, 결선 로드맵까지 구체화
가스계 손실률 계산	총중량 기준 손실률 계산	약제중량(총중량-공병중량) 기준으로 정정	미니어처 용기는 자체 무게 비중이 커 총중량 기준 계산이 실제 손실률을 과소평가 — 임계치 도달 예측이 4.48일에서 2.1일로 앞당겨짐(더 정확하고 더 보수적인 조기경보)
자탐 루프저항 판정	이동평균 기반 z-score 판정	절대임계값(1층)+선형회귀 추세(2층) 2층 판정	baseline이 서서히 상승하는 저항값을 따라가버려 "진짜 가속 열화"를 정상으로 오판하는 근본적 결함을 실측 중 발견해 전면 재설계. 오탐률 0% 달성
자탐1 온도 판정 기준	온도(°C) 환산값(temp_c) 기반 판정 예정	raw ADC 값 직접 판정, temp_c는 표시 전용으로 격하	손 온도계 대비 NTC 센서의 열적 이력현상(히스테리시스, raw 1000 이상 편차)을 실측으로 발견 — 환산 계수 오차에 영향받지 않는 raw 기반 판정으로 신뢰도 확보
앱의 역할	대시보드 축소판(모바일에서 같은 정보 보기)	독립적인 현장 점검·기록·증빙 도구(QR·점검완료·PDF)로 재정의	관제 거점 사용자(방재실)와 순회 점검 사용자(현장)의 요구사항이 근본적으로 다름을 설계 단계에서 인식 — 기존 대시보드 코드를 한 줄도 건드리지 않는 API 분리 구조로 병행 개발
AI 적용 범위	설계 초기에는 다수 설비에 AI 적용 검토	정확히 2곳(수계 총압펌프 분류, 자탐2 비화재보 판별)으로 의도적 한정	"여러 변수를 동시에 봐야 구분되는 문제"만 선별 적용하고 나머지는 통계·규칙 기반으로 구현 — 과설계를 피하고 각 판정의 설명가능성(explainability)을 확보

9-2. 정성적 개선: "이론 설계" → "실측 기반 근본원인 추적"

위 변경들을 관통하는 공통된 개선 패턴은, 초기 설계 단계의 이론적 가정을 실물 하드웨어·실측 데이터로 검증하면서 표면적 증상이 아니라 근본 원인까지 추적해 재설계했다는 점이다.

- 물리 원리 차원의 재검토 — CT 클램프의 DC 측정 불가(전자기 유도 원리), NTC 센서의 열적 이력현상, 로드셀의 영점(TARE) 오차와 배율 오차 구분 등, 표면적으로는 "센서가 이상하다"로 보일 수 있는 현상을 물리 원리 수준까지 파고들어 원인을 특정
- 죽은 코드·조건문 버그의 발견 — 수계 AI 가 "학습은 됐지만 실행되지 않고 있던" 사례처럼, 하드웨어 변경(INA219 제거)이 소프트웨어 조건문과 어긋나며 생긴 문제를 코드 리뷰가 아니라 실측 검증 과정에서 발견
- 정직한 판정 채택 — 수계 H-Q 성능시험에서 축소 모델의 한계를 임계값 완화로 감추지 않고, 이론값 그대로 두어 "정직한 불합격"이 나오도록 하고 그 한계를 발표자료에 명시하는 방향으로 확정
- 일정 단위의 완성도 추적 — 옛지별 완성도가 정량화되어 있어, "어느 부분이 결선 전까지 가장 시급한가"가 막연한 인상이 아니라 근거 있는 우선순위로 관리됨(예: 소화기 리프노드는 딥슬립 API 버그·위치독 미반영이 겹쳐 한때 가장 뒤쳐진 구간이었으나, 딥슬립 로직 자체를 제거하는 재설계로 두 문제를 한 번에 해소)

9-3. 종합 완성도 (2026-08-27 기준)

옛지	완성도	상태 요약
자탐1(차동식)	~95%	위치독·판정로직·점검모드 모두 확정 완료
자탐2(광전식)	~98%	위치독 반영 완료, 이상치 2건 제외 후 AI 재학습 완료
가스계(CO2)	~90%	위치독·캘리브레이션·임계값 확정. 온도드리프트 보정만 미해결(향후 보완 예정)
유도등	~95%	위치독 반영 완료, 4h 재시험은 선택사항으로 스킵 결정됨
소화기 게이트웨이	~85%	위치독 반영 완료, ESP-NOW/WiFi 채널 고정(Channel 6)
소화기 리프노드×4	~90%	딥슬립 로직 제거 + 상시 폴링·표준 위치독 적용으로 API 버그·위치독 미반영 모두 해소. 배터리 운용시간 재산정만 남음
수계(총압+주펌프)	~90%	위치독 반영, H-Q 곡선 판정 확정, AI 추론 버그 수정 완료

전체 평균은 대략 91~92% 수준으로 평가된다. 소화기 게이트웨이의 ESP-NOW/WiFi 채널 고정 코드 반영만 정리되면 하드웨어/펌웨어 단계는 사실상 마무리로 볼 수 있다.

10. 종합 향후 과제

10-1. 최우선 확인 항목

- 수계 — 체절 상태 전류가 실제 "과부하"에 해당하는 값인지 펌프(DC 모터) 특성상 실측 검증(AI 추론 경로 버그는 수정 완료)
- 가스계 — 온도 드리프트(영점 오차) 보정

10-2. 실배포 전 확인 필요 항목

- 가스계 공병중량 실측 캘리브레이션 — DEFAULT_EMPTY_CONTAINER_WEIGHT_G(현재 260.0g)를 설치 현장 실측값으로 교체
- 수계 RATED_PRESSURE_KPA 등 실측 데이터 수집 및 압력 자동추정 임계값 안정성 확인
- 각종 임계값(가속도 15.0m/s^2 , 확인대기 10 초, 압력 자동추정 임계 102%/20% 등) 실측 후 재조정
- 수계 ENVIRONMENT 를 "testbed"→"deployment"로 전환(성능시험 주기 6 시간→180 일), 가스계 전송주기를 시연용 5 초→실배포 24 시간으로 복귀
- 부저/LCD 실물 배선 및 GPIO 핀 확정, ESP32 실물 배선 후 boot_id/seq/NTP 동기화 확인
- 소화기 리프노드 배터리 운용시간 재산정 — 답슬립 제거로 상시 폴링 방식이 되면서 배터리 소모량이 늘어난 만큼, 실측 기반으로 충전 주기·배터리 용량이 실사용에 충분한지 확인 필요

10-3. 결선(11 월) 확장 로드맵

- 자탐 1/2, 수계, 가스계, 유도등, 소화기 게이트웨이 → RS-485(Modbus RTU) 유선 전환. 소화기 리프노드→게이트웨이 구간만 무선(ESP-NOW) 유지
- Heartbeat 기반 통신두절 감지, 로컬 플래시 버퍼링(SPIFFS/LittleFS) 등 Fail-Soft/Fail-Active 항목 결선 단계에서 구현
- 수계 CT 클램프 비접촉 전환(실제 소방펌프는 AC 구동이므로 적용 가능), 자탐 1 2 단계(접촉저항 열화) IR 온도센서 로드맵 실측 검증
- 앱: 대시보드-DB 연동으로 홈화면 상태 표시 실제화(최우선), /equipment/{id} 라우트 추가로 QR 필터링 진입점 완성, PDF 한글 폰트·점검이력 DB 영구저장 적용

11. 부록 — 참고 자료

- 펌웨어: server/fire_alarm_differential_node_v6.ino, fire_alarm_photoelectric_node_v3.ino, gas_node_v4.ino, evac_light_node_v5.ino, extinguisher_gateway_v3.ino, extinguisher_leafnode.ino, pump_node_INA219_v25.ino
- 서버 판정: bulchimbeon_sw/server/judge/differential.py, rules.py, classify.py, regression.py, test_mode.py, pump_performance_test.py, receiver/packet_parser.py
- 실험 데이터: data/차동식 엷지/, data/광전식 엷지/, data/가스계 엷지/, data/유도등 엷지/, data/수계 엷지/ 하위 CSV·분석 리포트 전체
- 설계 문서: docs/불침번_하드웨어_조감도_핀배치_v3.pdf, docs/Fail-Safe_설계철학_정리.md, docs/자탐 1(차동식)_실물실험_기록.md, docs/자탐 2(광전식)_실물실험_기록.md, docs/유도등_실물실험_기록.md, guides/수계_배관구조_최종가이드.md
- 팀 프로젝트 논의 기록(claude.ai 프로젝트 "불침번"):
자탐 1_온도상승률_판정로직_소프트웨어_인수인계.md, 자탐 1_온도_캘리브레이션_실험_설계.md, 가스계_캘리브레이션_결과_정리.md, 네트워크_안정화_대안_비교.md, ESP-NOW_WiFi_채널_불일치_진단.md, 결선_RS-485_유무선_하이브리드_아키텍처.md, 발표자료_구성_재검토.md
- 원본 문서: 불침번_하드웨어_최종기술문서_썬막.pdf, 불침번_소프트웨어_기술문서.md(v7), 앱개발_총정리.md, 불침번_프로젝트설명_사업확장성_개선방향.md

문서 끝 — 불침번 최종 기술문서(통합판)